

CSCE 645 GEOMETRIC MODELING · MODULE 02 · LECTURE NOTES

Meshes and Connectivity

The data structure every later algorithm is built on.

MODULE OUTCOMES

- Implement a half-edge structure from an indexed mesh.
- Write all the one-ring traversals.
- Handle boundaries correctly rather than by accident.
- Implement the basic topological operations: split, collapse, flip.
- Compute vertex normals and explain the weighting choice.

1 What an indexed mesh cannot do

The rendering representation — a vertex array plus a triangle index array — is exactly right for the GPU and useless for geometry processing. It records *which vertices form each face* and nothing about neighbourhood.

Question	Cost with an indexed mesh
Which faces are incident to vertex v ?	Scan all F faces. $O(F)$ per query, $O(VF)$ for a loop over all vertices.
Which face lies across edge (a,b) ?	Scan, or build a hash from edges to faces.
What are v 's neighbours, in rotational order?	Scan, then sort — and the ordering is exactly what curvature and the Laplacian need.
Is this mesh manifold?	Not directly answerable.

From Module 07 onward, every algorithm iterates over vertices and asks for the one-ring neighbourhood of each. An $O(F)$ query inside that loop is not viable, so a connectivity structure is required.

2 The half-edge structure

Represent each undirected edge as two directed **half-edges** pointing in opposite directions, one belonging to each adjacent face. Four references per half-edge suffice:

```

struct HalfEdge {
    int next;    // next half-edge around the same face
    int twin;    // the oppositely-directed half-edge of the same edge
    int vertex;  // the vertex this half-edge points TO
    int face;    // the face on its left
};

struct Vertex { int halfedge; vec3 position; }; // one outgoing half-edge
struct Face   { int halfedge; };               // one half-edge of the face

```

The invariants worth asserting during construction: $\text{twin}(\text{twin}(h)) == h$; following next from any half-edge returns to it after exactly the face's degree of steps; and $\text{face}(\text{next}(h)) == \text{face}(h)$. Checking these once at build time catches essentially every construction bug.

2.1 The traversals

```

// Faces around vertex v -- the one-ring.
int h0 = vertex[v].halfedge, h = h0;
do {
    visit(he[h].face);
    h = he[he[h].twin].next;    // twin, then next
} while (h != h0);

// Vertices of face f.
int h0 = face[f].halfedge, h = h0;
do { visit(he[h].vertex); h = he[h].next; } while (h != h0);

// The face across an edge.
int opposite = he[he[h].twin].face;

```

TWIN THEN NEXT

This is the move that rotates around a vertex, and it is not obvious until you draw it. Starting from an outgoing half-edge, twin gives you the incoming half-edge of the neighbouring face, and next then gives you the next outgoing half-edge around the same vertex.

Write it once, test it on a mesh where you know the answer, and never think about it again. Getting it wrong produces traversals that terminate but visit the wrong set — which is far harder to debug than an infinite loop.

With these, every adjacency query costs time proportional to the size of the answer, which is optimal.

3 Boundaries

A boundary edge has only one adjacent face, so one of its two half-edges has no face. There are two ways to represent this, and the choice has consequences.

Approach	How	Consequence
Null twins	A boundary half-edge has twin = -1 and no second half-edge exists.	Simple construction. <i>Every</i> traversal must test for -1, and the one you forget is the bug you ship.
Virtual boundary faces	Create one imaginary face per boundary loop, with genuine half-edges forming a cycle around the hole.	Slightly more complex construction; every half-edge has a valid twin, so traversal code has no special cases. Boundary detection becomes face < 0 or a flag.

CHOOSE VIRTUAL BOUNDARY FACES

The overwhelming majority of bugs in hand-written mesh libraries are boundary bugs, and most of them are a missing -1 check in a traversal written weeks after the structure.

Virtual boundary faces move the complexity into construction, where it is written once and tested once, instead of distributing it across every algorithm you will ever write.

Whichever you choose, choose it deliberately and document it. Mixing the two conventions in one codebase is worse than either.

4 Topological operations

Three local edits generate essentially all of mesh processing.

Operation	Effect	Used in
Edge split	Insert a new vertex on an edge, splitting both adjacent triangles in two. Net: +1 vertex, +3 edges, +2 faces.	Subdivision (Module 06), remeshing (Module 10).
Edge collapse	Merge the two endpoints into one, removing the edge and its two adjacent triangles. Net: -1 vertex, -3 edges, -2 faces.	Simplification (Module 10), LOD generation.
Edge flip	Rotate an edge to connect the two opposite vertices of its adjacent triangles instead. Counts unchanged.	Remeshing, Delaunay-ising, valence optimisation.

4.1 The preconditions on edge collapse

THE LINK CONDITION

Collapsing an edge (a, b) is topologically safe only when the intersection of a's one-ring neighbours and b's one-ring neighbours consists of exactly the two vertices opposite the edge.

If they share a third neighbour, collapsing fuses two parts of the surface that were not adjacent, producing a non-manifold vertex or a degenerate face. The mesh often still renders, and fails much later in something unrelated.

This is the standard guard and it is cheap: intersect two small sets.

A second precondition is geometric rather than topological: the collapse must not **flip** any adjacent face. Compute each affected face's normal before and after the proposed collapse and reject if any dot product goes negative. Without this check, simplification produces meshes with inverted triangles that shade black and self-intersect.

Edge flips need a precondition too: the flip must not create a duplicate edge, which happens when the two opposite vertices are already connected.

5 Normals

A face normal is unambiguous: the normalised cross product of two edge vectors, with the sign determined by winding order (CSCE 641 Module 02). A **vertex** normal is an average over incident faces, and the weighting is a real choice.

Weighting	Formula	Behaviour
Uniform	$\sum n_r$	Simple. Biased toward whichever side happens to have more triangles, regardless of their size — so it changes when you tessellate.

Weighting	Formula	Behaviour
Area-weighted	$\sum A_r n_r$	A large face contributes more. Better, and still tessellation-dependent: splitting one face into two does not change the total area but does change the sum if done unevenly.
Angle-weighted	$\sum \theta_r n_r$	Weight each face by its interior angle <i>at this vertex</i> . The angles around a vertex always sum to the same total regardless of how the faces are subdivided, so the result is tessellation-independent . Usually the right choice.

Tessellation independence is the property you actually want: subdividing a neighbouring face should not change a vertex's normal, because the surface did not change. Many engines use area weighting because it is marginally cheaper; angle weighting is better and the cost difference is negligible outside inner loops.

5.1 Render meshes and processing meshes are different

THIS CAUSES MORE CONFUSION THAN ANY OTHER PRACTICAL ISSUE

Smooth shading needs one normal per vertex. **Sharp** edges need the vertex duplicated, with each copy carrying a different normal — and the same applies at UV seams and material boundaries.

So a mesh exported for rendering has *more vertices* than the underlying surface has, and those duplicates are not connected in the connectivity structure. Loading such a file and building a half-edge mesh directly yields a surface full of cracks, with wrong curvature everywhere near a seam.

Weld first. Merge vertices by position (with a spatial hash and a tolerance), build connectivity on the welded manifold, process, and split again on export. Treating the render mesh and the processing mesh as separate objects is not pedantry — it is the only way this works.

6 What real meshes will do to you

Problem	Symptom	Repair
Duplicate vertices	Cracks; traversals that terminate early; wrong Euler characteristic.	Weld by position with a spatial hash and an epsilon. Do this first, always.
Non-manifold edges	Half-edge construction fails or produces a corrupt twin mapping.	Detect edges with more than two incident faces. Either cut the surface along them or reject the mesh with a clear message.
Inconsistent winding	Face normals point in random directions; lighting is patchy.	Breadth-first traversal across shared edges, flipping faces to agree with their neighbour. Works on any orientable surface.
Zero-area triangles	NaN normals that propagate everywhere.	Remove degenerate faces before building connectivity. Check for NaN after every normal computation while developing.

Problem	Symptom	Repair
Isolated vertices	Euler characteristic wrong; vertices with no half-edge.	Remove, or at least detect and report.

All five repairs are required in Project 1 because every subsequent module assumes clean input. Building them once, with tests, is the difference between a library you can rely on and one that fails mysteriously in Module 09.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Keenan Crane — DDG, Combinatorial Surfaces and the half-edge lecture	The structure, the invariants, and the traversals, with the topology made precise.
libigl tutorial — adjacency and traversal	How a production library does this with flat index arrays rather than a pointer structure, and why.
OpenMesh documentation	A mature half-edge implementation. Read the handle and circulator design before writing your own API.
Max Wardetzky et al. — 'Discrete Laplace operators: no free lunch' (free)	Not needed until Module 08, but the normal-weighting discussion here is the same kind of argument — worth seeing early.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a half-edge structure built from an indexed triangle mesh. Assert all three invariants after construction and verify on a cube, a sphere, and a mesh with boundary.
- 2 Implement virtual boundary faces. Verify that a disc-topology mesh has exactly one boundary loop and that every half-edge has a valid twin.
- 3 Write the one-ring traversal and verify it visits the correct number of faces for every vertex of a known mesh. Then verify the average count is about 6.
- 4 Implement edge split, edge collapse, and edge flip, each with their preconditions. Write a test that performs a thousand random legal operations and re-validates the mesh afterwards.
- 5 Deliberately perform a collapse that violates the link condition. Visualise the resulting non-manifold vertex and describe how you would detect it after the fact.

- 6 Implement all three vertex normal weightings. Construct a mesh where uniform and angle weighting differ visibly, render all three, and explain which is correct and why.
- 7 Write the five mesh repairs from §6 and run them on a downloaded scanned model. Report what each one found.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What question can an indexed mesh not answer efficiently, and why does it matter from Module 07 onward?
- 2 List the four fields of a half-edge and state two invariants.
- 3 What does the sequence twin-then-next accomplish?
- 4 Give both boundary representations and say which this course recommends, with a reason.
- 5 State the link condition and say what goes wrong without it.
- 6 Why is angle weighting preferable to area weighting for vertex normals?
- 7 Why must a mesh loaded from an OBJ file usually be welded before processing?